

UNIVERSITÀ POLITECNICA DELLE MARCHE

INGEGNERIA INFORMATICA E
DELL'AUTOMAZIONE

Corso di
DATA SCIENCE

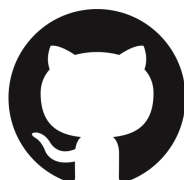


**BiblioBot: Un chatbot conversazionale per
la ricerca di libri basato su Rasa**



Docenti:

Domenico Ursino
Christofer Buratti



Studenti:

Matteo Copertari
Kevin Giusti
Matteo Stronati
Jacopo Tarulli

Anno accademico 2025-2026

Indice

1	Introduzione	4
1.1	Descrizione del progetto	4
1.2	Tecnologie Utilizzate	5
1.2.1	Rasa Framework	5
1.2.2	Gestione dell'Ambiente: Anaconda	5
1.2.3	Esposizione del servizio: Ngrok	6
1.2.4	Python e Pandas	6
1.3	Struttura del Progetto	6
2	Scelta del Dataset e pulizia	8
2.1	Analisi del Dataset Originale	8
2.2	Pipeline di Pulizia Testuale Avanzata	9
2.2.1	Rimozione della Stringa "Garbage" nei Generi	9
2.2.2	Correzione delle Descrizioni Troncare	9
2.3	Filtraggio delle Righe e Deduplica	9
2.3.1	Deduplica Basata sulla Popolarità	10
2.3.2	Filtro di Rilevanza e Qualità	10
2.3.3	Normalizzazione e Filtro Linguistico	10
2.4	Selezione e Trasformazione delle Colonne (Feature Engineering)	11
2.4.1	Colonne Rimosse	11
2.4.2	Nuove Features Generate	11
2.4.3	Schema del Dataset Finale	11
3	Implementazione del Bot	13
3.1	Intents: classificazione delle intenzioni dell'utente	13
3.2	Entities: estrazione di parametri strutturati	16
3.3	Slots: variabili di stato del dialogo	17
3.4	Repsponses: templates di risposta statica	19
3.5	Stories: percorsi conversazionali del Bot	19
3.6	Rules: gestione di comportamenti fissi	21
3.7	Actions e reset degli slots	22
4	Setup Telegram e avvio del Bot	25
4.1	Installazione dell'ambiente di sviluppo	25
4.2	Creazione del Bot con BotFather	26
4.3	Configurazione di ngrok per il webhook	27
4.4	Configurazione del file <code>credentials.yml</code>	28

4.5	Avvio del sistema	29
5	Test del Bot	30
5.1	Funzionalità verificate	30
5.2	Impostazione dei test	30
5.3	Casi di test eseguiti	31
5.3.1	Avvio	31
5.3.2	Ricerca per titolo	31
5.3.3	Ricerca per filtri	32
5.4	Conclusioni sui test	35

Elenco delle figure

3.1	Lista degli intents definiti nel file <code>domain.yml</code>	14
3.2	Esempi di training data per alcuni intents nel file <code>nlu.yml</code>	15
3.3	Lista delle entities definite nel file <code>domain.yml</code>	16
3.4	Esempi di definizione degli slots nel file <code>domain.yml</code>	18
3.5	Esempi di responses definite nel file <code>domain.yml</code>	19
3.6	Esempi di stories definite nel file <code>stories.yml</code>	20
3.7	Rules definite nel file <code>rules.yml</code>	21
3.8	Lista delle actions definite	23
4.1	Processo di creazione di "Biblio2026_bot" tramite BotFather	27
4.2	Connessione con ngrok con url evidenziato	28
5.2	Risultato della ricerca per titolo	32
5.3	Esempio di ricerca tramite filtro per genere	33
5.4	Esempi di ricerca con filtri quantitativi	34
5.5	Esempi di ricerca con filtri testuali	34
5.6	Avvio di una nuova ricerca	35

1 Introduzione

L'evoluzione dell'interazione uomo-macchina ha subito una rapida accelerazione negli ultimi decenni, spostandosi progressivamente dalle interfacce a riga di comando (CLI) e grafiche (GUI) verso le cosiddette interfacce conversazionali (CUI). In questo contesto, i *chatbot* e gli assistenti virtuali rappresentano una delle applicazioni più promettenti dell'Intelligenza Artificiale, offrendo agli utenti la possibilità di interagire con i sistemi informatici utilizzando il linguaggio naturale.

Il presente elaborato descrive la progettazione e lo sviluppo di un assistente conversazionale verticale, specializzato nel dominio letterario. L'obiettivo del progetto è fornire uno strumento intelligente in grado di assistere l'utente nella ricerca di libri all'interno di un vasto catalogo, superando i limiti dei tradizionali motori di ricerca basati su parole chiave e offrendo un'esperienza simile a quella di un consiglio fornito da un libraio esperto.

1.1 Descrizione del progetto

La quantità di opere letterarie disponibili sul mercato è cresciuta esponenzialmente, creando un fenomeno noto come *information overload*. Per un lettore, individuare il prossimo libro da leggere può diventare un compito arduo di fronte a migliaia di opzioni. I sistemi di raccomandazione tradizionali spesso si basano su filtri statici o su algoritmi "black-box" che non permettono all'utente di esplicitare le proprie preferenze in modo discorsivo.

Il chatbot realizzato in questo progetto si propone di colmare questa lacuna, permettendo un'interazione fluida e granulare. A differenza di una semplice barra di ricerca, il bot è in grado di comprendere richieste complesse e multicriterio, come ad esempio:

"I want to read a thriller book under 300 pages"

Il sistema è stato progettato per interrogare una base di conoscenza strutturata (descritta nel Capitolo 2), filtrando i risultati in base a parametri specifici quali:

- **Genere letterario:** classificando le richieste dell'utente in macro-categorie.
- **Autore:** identificando nomi di scrittori anche all'interno di frasi discorsive.
- **Caratteristiche fisiche:** filtraggio per numero di pagine.
- **Metriche di qualità:** selezione basata sul rating medio degli utenti calcolato su migliaia di recensioni.

- **Personaggi:** ricerca basata sulla presenza di specifici personaggi nella trama.

L'architettura del sistema combina tecniche di *Natural Language Understanding* (NLU) per l'interpretazione semantica della richiesta e logiche di *Information Retrieval* per l'estrazione puntuale dei dati dal dataset pulito.

1.2 Tecnologie Utilizzate

Per lo sviluppo del chatbot è stato selezionato uno stack tecnologico moderno e open-source, privilegiando strumenti che garantissero modularità e controllo completo sul flusso di conversazione.

1.2.1 Rasa Framework

Il cuore del sistema è **Rasa**, un framework open-source leader per lo sviluppo di AI conversazionali. La scelta di Rasa, rispetto a soluzioni proprietarie in cloud (come Google Dialogflow o Amazon Lex), è stata dettata da diversi fattori chiave:

- **Privacy e Controllo:** Rasa permette di eseguire l'intero stack in locale (*on-premise*), senza dover inviare i dati delle conversazioni a terze parti.
- **Architettura Modulare:** Rasa separa nettamente la componente di comprensione del linguaggio (**Rasa NLU**) dalla gestione del dialogo (**Rasa Core**). Questo permette di affinare indipendentemente la capacità del bot di capire l'intento dell'utente e la sua strategia di risposta.
- **Custom Actions:** La possibilità di scrivere codice Python personalizzato per eseguire azioni complesse (come il filtraggio di un DataFrame Pandas) è nativa in Rasa tramite il componente *Action Server*.

1.2.2 Gestione dell'Ambiente: Anaconda

Data la complessità delle dipendenze delle librerie di Machine Learning (come TensorFlow, su cui Rasa si basa), è stato fondamentale isolare l'ambiente di sviluppo. È stato utilizzato **Conda** (distribuzione Anaconda/Miniconda) per creare un *virtual environment* dedicato. Questo approccio ha permesso di:

- Gestire una versione specifica di Python (3.8/3.9) compatibile con Rasa 3.x.
- Installare le librerie necessarie per la manipolazione dati (**pandas**, **numpy**) senza conflitti con altre installazioni di sistema.
- Garantire la riproducibilità del progetto su macchine diverse esportando il file `environment.yml`.

1.2.3 Esposizione del servizio: Ngrok

Durante la fase di sviluppo e testing, è stato utilizzato **Ngrok**, un tool che permette di esporre un server locale verso internet tramite un tunnel sicuro. Nel contesto di questo progetto, Ngrok è essenziale per collegare il server Rasa (che gira in locale sulla porta 5005) a interfacce esterne o canali di test (come Rasa X, Telegram o interfacce web custom) che necessitano di un endpoint HTTPS pubblico per comunicare tramite webhook.

1.2.4 Python e Pandas

Il linguaggio di programmazione utilizzato per la logica applicativa è **Python**. In particolare, la libreria **Pandas** gioca un ruolo cruciale all'interno delle *Custom Actions*: ogni volta che il bot deve cercare un libro, il codice Python carica il dataset processato, applica i filtri corrispondenti agli *entities* estratti (genere, rating, ecc.) e restituisce i risultati al motore di dialogo.

1.3 Struttura del Progetto

Il progetto segue la struttura standard imposta dal comando `rasa init`, arricchita dai file necessari per la gestione del dataset. Di seguito vengono descritti i file e le directory principali che compongono la soluzione:

- **data/**: Cartella contenente i dati di addestramento per il modello di Machine Learning.
 - **nlu.yml**: Definisce gli *intents* (intenzioni dell'utente) e fornisce frasi di esempio annotate con le relative *entities*.
 - **stories.yml**: Contiene esempi di flussi di conversazione completi, utilizzati per addestrare il modello di gestione del dialogo (*Dialogue Management*).
 - **rules.yml**: Definisce regole fisse (comportamenti deterministici) che il bot deve seguire in specifiche situazioni (es. saluti, fallback, attivazione di form).
- **actions/**: Directory dedicata alla logica custom.
 - **actions.py**: Script Python che contiene l'implementazione delle classi **Action**, responsabili dell'interrogazione del dataset e della logica di business.
- **domain.yml**: Rappresenta il "mondo" del chatbot. In questo file vengono dichiarati tutti gli intent, le entity, gli slot (memoria del bot), le action e le risposte testuali predefinite (*templates/responses*).
- **config.yml**: File di configurazione della pipeline NLU e delle policies del modello Core. Qui vengono specificati i tokenizzatori, i featurizzatori e gli algoritmi di classificazione (es. DIETClassifier).

- `endpoints.yml`: Configurazione dei collegamenti verso servizi esterni, in particolare per mettere in comunicazione il server Rasa Core con l'Action Server.
- `goodreads_chatbot_cleaned.csv`: Il dataset ripulito (frutto del lavoro descritto nel Capitolo 2), utilizzato come knowledge base per le ricerche.

L'organizzazione modulare dei file permette una chiara separazione delle responsabilità: i file nella cartella `data/` istruiscono il modello su "cosa capire" e "come comportarsi", mentre la cartella `actions/` definisce "cosa fare" concretamente quando viene richiesta un'operazione sui dati.

2 Scelta del Dataset e pulizia

La qualità delle risposte fornite dal chatbot dipende intrinsecamente dalla qualità dei dati sottostanti. Per questo progetto, non è stato sufficiente utilizzare il dataset grezzo scaricato da Kaggle, in quanto presentava numerose imperfezioni strutturali, ridondanze e informazioni non pertinenti per un'interazione conversazionale. È stata quindi sviluppata una pipeline di preprocessing dedicata, implementata in un ambiente **Jupyter Notebook** utilizzando la libreria **Pandas**, con l'obiettivo di distillare un corpus di dati pulito, coerente e ottimizzato per le performance del framework Rasa.

In questo capitolo vengono descritte in dettaglio le operazioni di pulizia (*Data Cleaning*), la selezione degli attributi (*Feature Selection*) e le trasformazioni applicate (*Feature Engineering*), analizzando quantitativamente la riduzione del dataset in termini di righe e colonne.

2.1 Analisi del Dataset Originale

Il dataset di partenza, denominato *Goodreads Books Dataset*¹, è costituito da dati eterogenei raccolti tramite *web scraping*. Allo stato grezzo, il file `goodreadss_books.csv` conteneva **19.509 righe** e **22 colonne**, è costituito da dati eterogenei raccolti tramite *web scraping*. Allo stato grezzo, il file `goodreadss_books.csv` conteneva **19.509 righe** e **22 colonne**. Sebbene ricco di informazioni, presentava diverse criticità:

- **Duplicati Semantici:** Lo stesso libro appariva più volte a causa delle diverse edizioni (copertina rigida, tascabile, ebook, edizioni speciali) o a causa di lievi variazioni nel titolo (ad esempio l'uso di maiuscole diverse o della & commerciale al posto dell'avverbio 'and'), gonfiando artificialmente il dataset.
- **Rumore e Artefatti di Scraping:** Le colonne testuali (descrizioni e generi) presentavano errori sistematici di formattazione e ripetizioni di stringhe spurie, descritti in dettaglio nella sezione successiva.
- **Attributi Superflui:** Colonne come gli ISBN o il conteggio dettagliato dei voti per ogni stella (1-5) occupavano memoria senza offrire valore aggiunto alla conversazione.
- **Inconsistenza Linguistica:** La lingua era indicata con codici eterogenei (es. "en-US", "eng", "en-GB") e includeva opere in lingue non supportate dal modello NLU.

¹<https://www.kaggle.com/datasets/khanhdnguyen/goodreadss-books>

2.2 Pipeline di Pulizia Testuale Avanzata

Prima di procedere al filtraggio delle righe, è stato necessario sanare il contenuto delle colonne **genres** e **description**, che risultavano pesantemente compromesse da errori generati dal crawler originale. Questa fase è stata critica per garantire che il chatbot non mostrasse output sgrammaticati o ripetitivi.

2.2.1 Rimozione della Stringa "Garbage" nei Generi

L'analisi della colonna **genres** ha evidenziato un'anomalia sistematica: all'inizio della lista dei generi di quasi ogni libro, veniva ripetuta per due volte una lunga stringa fissa contenente un elenco alfabetico di tutte le categorie possibili. La stringa in questione, definita come "garbage string", era la seguente:

"Art,Biography,Business,Children's,...,Sports,Thriller,Travel,Young Adult"

Questa ripetizione mascherava i generi reali dell'opera, rendendo impossibile una corretta classificazione. La pipeline di pulizia ha provveduto a rimuovere chirurgicamente questa sottostringa dall'inizio di ogni campo, lasciando emergere solo i tag pertinenti (es. solo "Fantasy, Young Adult" invece dell'intero elenco standard).

2.2.2 Correzione delle Descrizioni Troncare

Un secondo artefatto significativo riguardava la colonna **description**. È stato osservato che per le descrizioni superiori a una certa lunghezza (circa 300 caratteri), il testo presentava una duplicazione parziale all'inizio. Nello specifico, i primi ≈ 300 caratteri della trama venivano riportati, troncati improvvisamente, e seguiti immediatamente dal testo completo della descrizione che ricominciava da capo.

Esempio del pattern rilevato:

[Inizio Trama Troncato...] [Inizio Trama... Trama Completa]

Per correggere questo errore, è stata implementata una logica di controllo sulle stringhe che identifica il punto di ripetizione e rimuove il prefisso troncato, ripristinando la fluidità e la correttezza grammaticale della sinossi. Senza questo intervento, il chatbot avrebbe riportato agli utenti descrizioni spezzate e ridondanti.

2.3 Filtraggio delle Righe e Deduplica

Una volta risanati i testi, il processo ha seguito un approccio "a imbuto", applicando filtri sequenziali per rimuovere i record di bassa qualità o duplicati.

2.3.1 Deduplica Basata sulla Popolarità

Il problema principale del dataset originale era la presenza di molteplici record per lo stesso titolo. Mantenere i duplicati avrebbe confuso il chatbot. È stata applicata una logica di deduplica "*Survival of the Fittest*" (Sopravvivenza del più adatto): raggruppando i libri per titolo e autore, è stato conservato esclusivamente il record con il maggior numero di interazioni (somma di `num_ratings` e `num_reviews`).

- **Impatto:** Rimozione di **2.603** righe (duplicate).
- **Residuo:** 16.906 libri.

2.3.2 Filtro di Rilevanza e Qualità

Un sistema di raccomandazione deve suggerire item di valore. I libri con pochissime valutazioni spesso presentano metadati incompleti. È stato imposto un vincolo minimo di popolarità:

$$\text{Keep if } (\text{num_ratings} \geq 10) \wedge (\text{num_reviews} \geq 20)$$

Questa soglia ha permesso di eliminare opere di nicchia o con dati statisticamente non significativi.

- **Impatto:** Rimozione di ulteriori **3.733** righe.
- **Residuo:** 13.173 libri.

2.3.3 Normalizzazione e Filtro Linguistico

Poiché il chatbot è addestrato in lingua inglese, era necessario garantire la coerenza linguistica. La colonna `language` è stata prima normalizzata (es. "en-US", "en-CA" → "English"). Successivamente, sono stati scartati tutti i libri in lingue diverse dall'inglese, mantenendo però le categorie "Unknown" e "Multiple Languages" per evitare falsi negativi.

- **Impatto:** Rimozione di **793** righe.
- **Residuo Finale:** **12.380** libri.

La Tabella 2.1 riassume la riduzione progressiva del volume dei dati, evidenziando come circa il **36%** del dataset originale sia stato scartato.

Fase di Elaborazione	Righe Rimosse	Righe Rimanenti
Dataset Grezzo	-	19.509
1. Deduplica Titoli	-2.603	16.906
2. Filtro Popolarità (Ratings/Reviews)	-3.733	13.173
3. Filtro Lingua	-793	12.380

Table 2.1: *Funnel di pulizia del dataset: dai dati grezzi al file finale*

2.4 Selezione e Trasformazione delle Colonne (Feature Engineering)

Parallelamente alla riduzione delle righe, è stata effettuata una selezione delle colonne (*features*).

2.4.1 Colonne Rimosse

Sono state rimosse dal dataset finale circa **8 colonne** considerate non essenziali:

- **Identificatori tecnici:** `isbn` e `isbn13`.
- **Dettagli granulari dei voti:** Le 5 colonne relative al conteggio specifico dei voti (es. `rated_5`, `rated_4`) sono state eliminate, mantenendo solo la media e il totale.
- **Luoghi:** La colonna `places` conteneva dati troppo sparsi.

2.4.2 Nuove Features Generate

Per migliorare le capacità del bot, sono state create nuove colonne:

- **Annalità (year):** Estratto l'anno dalla data di pubblicazione originale (`publish_date`) per facilitare le query temporali.
- **Gestione delle Serie (series_name, series_vol):** La colonna originale `series` è stata parsata per separare il nome della saga dal numero del volume (es. da "Harry Potter #6" a "Harry Potter" e "6").

2.4.3 Schema del Dataset Finale

Il risultato è il file `goodreads_chatbot_cleaned.csv`, composto da 15 colonne ottimizzate:

- **Metadati:** `bookId`, `title`, `author`, `year`, `language`.
- **Contenuto:** `description`, `genres`, `characters`, `awards`, `num_pages`.
- **Metriche:** `avg_rating`, `num_ratings`, `num_reviews`.
- **Serie:** `series_name`, `series_vol`.

Table 2.2: *Campione del dataset - Parte 1: Metadati di Identificazione e Serie. La colonna Series originale è stata divisa in Series Name e Vol.*

ID	Title	Author	Year	Series Name	Vol
14489	\$30 Film School	Michael W. Dean	2006	-	-
4912	'Tis	Frank McCourt	1999	Frank McCourt	2
19594	'What Color Is Your...	R. N. Bolles	2005	-	-
15979	'Where Are You Going...	Joyce Carol Oates	1994	-	-
8876	*Planet of the Apes*...	Eric Greene	1999	-	-

Table 2.3: *Campione del dataset - Parte 2: Contenuto e Metriche (riferite agli stessi ID della Tabella 2.2). Si noti la pulizia dei generi e la normalizzazione della lingua.*

ID	Genres (Partial)	Lang	Pages	Avg Rating	Votes	Awards (Partial)
14489	Culture, Film, Nonfiction	Eng	544	3.48	82	-
4912	Nonfiction, Autobiography	Eng	495	3.69	57,714	Audie Award...
19594	Business, Self Help	Eng	45	3.71	592	-
15979	Short Stories, Horror	Eng	176	4.11	1,823	-
8876	Nonfiction, Media Tie In	Eng	286	3.91	56	-

3 Implementazione del Bot

Questo capitolo descrive l'implementazione tecnica del chatbot conversazionale per la ricerca di libri, realizzato utilizzando il framework **Rasa**. L'obiettivo è stato quello di creare un assistente virtuale in grado di comprendere richieste in linguaggio naturale e fornire raccomandazioni personalizzate basate su criteri multipli quali genere, autore, personaggi, numero di pagine e valutazione media.

L'implementazione si articola seguendo l'**architettura modulare** di Rasa, che separa la comprensione del linguaggio naturale (**Natural Language Understanding, NLU**) dalla gestione del dialogo (**Dialogue Management**) e dalla logica applicativa (**Custom Actions**). Questa separazione permette di sviluppare, testare e modificare ogni componente in modo indipendente, garantendo scalabilità e manutenibilità del sistema.

Nelle sezioni seguenti verranno analizzati in dettaglio i componenti fondamentali del bot. Si partirà dalla definizione degli **intents** (Sezione 3.1) e delle **entities** (Sezione 3.2), elementi chiave per l'interpretazione delle intenzioni dell'utente e l'estrazione di informazioni rilevanti. Successivamente verranno descritti gli **slots** (Sezione 3.3), utilizzati per memorizzare lo stato della conversazione, e le **responses** (Sezione 3.4), che definiscono i messaggi predefiniti del bot. Il flusso conversazionale viene gestito attraverso **stories** (Sezione 3.5) e **rules** (Sezione 3.6), che definiscono rispettivamente i percorsi di dialogo tipici e le regole deterministiche da applicare in situazioni specifiche. Infine, la Sezione 3.7 illustra le **custom actions**, ovvero la logica applicativa che implementa la ricerca nel dataset, il filtraggio dei risultati e la gestione dello stato della conversazione attraverso il **reset degli slot**.

3.1 Intents: classificazione delle intenzioni dell'utente

Gli **intents** rappresentano le intenzioni dell'utente, ovvero lo scopo che si cela dietro il messaggio inviato al chatbot. In Rasa, ogni intent viene associato a un insieme di frasi di esempio che permettono al modello NLU di classificare correttamente nuovi messaggi ricevuti dagli utenti. La definizione degli intents costituisce il primo passo fondamentale per consentire al bot di comprendere cosa l'utente desidera ottenere dalla conversazione.

Nel contesto di questo chatbot per la ricerca di libri, sono stati definiti gli intents mostrati nella Figura 3.1, ciascuno dei quali corrisponde a una specifica funzionalità di **ricerca** o **interazione** con il sistema.

```
intents:
- greet
- goodbye
- affirm
- deny
- stop
- new_search
- find_book
- specify_genre
- find_book_by_title
- find_book_by_author
- find_book_by_page_range
- find_book_by_page
- find_book_by_rating_range
- find_book_by_character
- nlu_fallback
```

Figure 3.1: *Lista degli intents definiti nel file `domain.yml`*

Gli intents implementati si dividono, principalmente, in due categorie:

Intents di gestione conversazionale

- `greet`: saluto iniziale dell'utente.
- `goodbye`: congedo finale dall'utente.
- `affirm`: conferma o risposta affermativa.
- `deny`: negazione o risposta negativa.
- `stop`: interruzione della conversazione.
- `new_search`: richiesta di iniziare una nuova ricerca azzerando i filtri precedenti.
- `nlu_fallback`: gestione dei messaggi non compresi dal sistema.

Intents di ricerca

- `find_book`: ricerca generica di libri.
- `specify_genre`: ricerca specificando il genere del libro.
- `find_book_by_title`: ricerca del libro per titolo.
- `find_book_by_author`: ricerca del libro per autore.

- `find_book_by_page_range`: ricerca del libro per range di pagine.
- `find_book_by_page`: ricerca del libro per numero esatto di pagine.
- `find_book_by_rating_range`: ricerca del libro per valutazione minima.
- `find_book_by_character`: ricerca del libro per personaggio presente.

Ogni intent di ricerca è associato a specifiche entities che permettono di estrarre i parametri necessari per filtrare il dataset dei libri, come descritto nella sezione successiva.

Per addestrare il modello NLU a riconoscere gli intents, è necessario fornire esempi rappresentativi di frasi per ciascuno di essi nel file `nlu.yml`. È fondamentale garantire varietà negli esempi, includendo diverse formulazioni e strutture sintattiche, affinché il modello possa generalizzare su messaggi mai visti prima. La Figura 3.2 mostra alcuni esempi di training data per tre intents significativi: `new_search`, `specify_genre` e `find_book_by_author`.

```
- intent: new_search
  examples: |
    - start over
    - new search
    - search again
    - i want to do a new search
    - reset
    - clear filters
    - I want to search for something else
    - start a new search
    - forget everything
    - clear all

- intent: specify_genre
  examples: |
    - I like [fiction](genre)
    - something in [fantasy](genre)
    - a [mystery](genre) book
    - [romance](genre) novels
    - I prefer [thriller](genre)
    - looking for [horror](genre)
    - [science fiction](genre) please
    - a [historical fiction](genre) would be nice
    - I enjoy [young adult](genre) books
    - [classics](genre) literature
    - [nonfiction](genre) books
    - something about [history](genre)
    - I want a book talking about [sports](genre)
    - I want a book with genre [fantasy](genre)
    - looking for a [drama](genre) genre book
    - genre [adventure](genre)
    - I'm interested in [biography](genre) genre
    - can you find [comedy](genre) genre books
    - show me [crime](genre) genre
    - [poetry](genre) genre please
    - books in the [philosophy](genre) genre
    - I'd like the genre [self-help](genre)
    - find me [autobiography](genre) genre

- intent: find_book_by_author
  examples: |
    - Find books by [Stephen King](author)
    - Show me titles written by [Agatha Christie](author)
    - Search for books from [George Orwell](author)
    - books by the author [Italo Calvino](author)
    - Do you have any books by [J.K. Rowling](author)?
    - List novels by [Haruki Murakami](author)
    - Find all books authored by [Jane Austen](author)
    - work written by [Isaac Asimov](author)
    - Show books by [Brandon Sanderson](author)
    - What books did [Neil Gaiman](author) write?
    - Find books written by [Toni Morrison](author)
    - Get me books from [Gabriel Garcia Márquez](author)
    - Show me novels by [Umberto Eco](author)
    - I want books by [Charles Dickens](author)
    - Do you have [Mark Twain](author) novels?
    - A book by the author [Virginia Woolf](author)
    - Show works of [Fyodor Dostoevsky](author)
    - books written by [C.S. Lewis](author)
    - I need books by [Roald Dahl](author)
    - Find me something by [Gillian Flynn](author)
    - author [Dan Brown](author)
    - Show me works by [Ken Follett](author)
    - books authored by [Isabel Allende](author)
    - the author [Paulo Coelho](author)
    - novels by the author [John Green](author)
    - [Brandon Sanderson](author) written books
    - [Margaret Atwood](author) author
```

Figure 3.2: Esempi di training data per alcuni intents nel file `nlu.yml`

Le frasi includono annotazioni delle entities (indicate tra parentesi quadre con il formato `[valore](entity_name)`), che permettono al modello di apprendere contem-

poraneamente sia l'intent che le informazioni da estrarre. Ad esempio, "I like [fiction](genre)" permette di riconoscere l'intenzione di specificare un genere e il valore "fiction". Per ogni intent sono state definite circa 20-30 varianti, coprendo formulazioni esplicite come "Find books by [Stephen King](author)" e colloquiali come "Do you have any books by [J.K. Rowling](author)?". L'intent `new_search`, che non prevede entities, serve per azzerare lo stato conversazionale con frasi come "start over" o "clear filters".

3.2 Entities: estrazione di parametri strutturati

Le **entities** rappresentano informazioni specifiche e strutturate estratte dalle frasi degli utenti. Mentre gli intents identificano l'intenzione generale del messaggio, le entities catturano i dettagli rilevanti necessari per soddisfare la richiesta. Nel contesto di un chatbot per la ricerca di libri, le entities corrispondono ai parametri di filtraggio che l'utente desidera applicare, come il genere letterario, il nome dell'autore o il numero di pagine. L'estrazione delle entities avviene attraverso annotazioni manuali negli esempi di training presenti nel file `nlu.yml`. Durante la fase di training, il modello NLU apprende a riconoscere pattern simili in nuovi messaggi, estraendo automaticamente le informazioni rilevanti anche da formulazioni diverse da quelle viste negli esempi.

La Figura 3.3 mostra l'elenco completo delle entities definite per questo chatbot, dichiarate nel file `domain.yml`.

```
entities:
  - genre
  - title
  - author
  - num_pages
  - min_num_pages
  - max_num_pages
  - min_avg_rating
  - characters
  - book_index
```

Figure 3.3: *Lista delle entities definite nel file `domain.yml`*

Le entities implementate consentono di filtrare il dataset contenente i libri secondo diversi criteri:

- **genre:** genere letterario (es. Fantasy, Mystery, Romance).
- **title:** titolo o parte del titolo del libro.

- `author`: nome dell'autore.
- `num_pages`: numero esatto di pagine.
- `min_num_pages`: numero minimo di pagine.
- `max_num_pages`: numero massimo di pagine.
- `min_avg_rating`: valutazione minima richiesta.
- `characters`: nome di un personaggio presente nel libro.
- `book_index`: indice numerico utilizzato per selezionare un libro specifico da una lista di risultati.

Le entities estratte vengono successivamente memorizzate negli slots, come spiegato nella Sezione 3.3, per mantenere lo stato della conversazione e permettere ricerche incrementalmente con filtri multipli.

3.3 Slots: variabili di stato del dialogo

Gli **slots** rappresentano la memoria del chatbot, ovvero le variabili utilizzate per memorizzare informazioni durante la conversazione. Mentre le entities vengono estratte dai messaggi degli utenti, gli slots persistono attraverso i turni di dialogo, mantenendo lo stato della conversazione e permettendo al bot di ricordare informazioni precedentemente fornite. In Rasa, gli slots possono influenzare o meno il comportamento del dialogue manager attraverso il parametro `influence_conversation`. Nel caso di questo chatbot, tutti gli slots hanno `influence_conversation: false`, poiché il flusso conversazionale non deve dipendere dai valori specifici memorizzati, ma solo dagli intents riconosciuti. Gli slots vengono utilizzati esclusivamente per accumulare i criteri di ricerca e passarli alle custom actions che interrogano il dataset.

La Figura 3.5 mostra esempi di definizione degli slots nel file `domain.yml`, suddivisi in due categorie: a sinistra alcuni slots per i filtri di ricerca, a destra quelli per la gestione dello stato conversazionale.

```

slots:
  genre:
    type: text
    influence_conversation: false
    mappings:
      - type: from_entity
        entity: genre

  title:
    type: text
    influence_conversation: false
    mappings:
      - type: from_entity
        entity: title

  author:
    type: text
    influence_conversation: false
    mappings:
      - type: from_entity
        entity: author

  num_pages:
    type: float
    influence_conversation: false
    mappings:
      - type: from_entity
        entity: num_pages

found_specific_book:
  type: bool
  influence_conversation: true
  mappings:
    - type: custom
  initial_value: false

book_index:
  type: text
  influence_conversation: false
  mappings:
    - type: from_entity
      entity: book_index

suggested_books:
  type: any
  influence_conversation: false
  mappings:
    - type: custom

```

Figure 3.4: Esempi di definizione degli slots nel file *domain.yml*

Nella parte sinistra dell'immagine sono rappresentati alcuni degli slots utilizzati per memorizzare i criteri di ricerca specificati dall'utente, come `genre`, `title`, `author` e `num_pages`. Questi slots hanno tutti tipo `text` o `float` e sono mappati direttamente dalle corrispondenti entities attraverso il mapping `from_entity`. Nella parte destra sono invece mostrati slots più specifici per la gestione dello stato del dialogo:

- `found_specific_book` (`bool`): indica se è stato trovato un libro specifico corrispondente alla ricerca. Questo slot ha `influence_conversation: true` poiché il suo valore determina se il bot deve mostrare i dettagli di un singolo libro o una lista di suggerimenti.
- `book_index` (`text`): memorizza l'indice numerico del libro selezionato dall'utente quando viene presentata una lista di risultati. Anche questo utilizza il mapping `from_entity` per catturare il numero specificato dall'utente.
- `suggested_books` (`text`): contiene la lista dei libri suggeriti in formato serializzato, permettendo al bot di reperirla nelle azioni successive.

Gli slots vengono popolati automaticamente quando le corrispondenti entities vengono estratte dai messaggi dell'utente e possono essere azzerati tramite comandi e azioni specifiche che verranno mostrate nella Sezione 3.7.

3.4 Responses: templates di risposta statica

Le **responses** sono messaggi predefiniti che il bot invia all'utente in situazioni specifiche del dialogo. A differenza dei messaggi generati dinamicamente dalle custom actions, le responses sono statiche e vengono definite direttamente nel file `domain.yml`. Ogni response può avere multiple varianti e Rasa ne seleziona casualmente una per rendere la conversazione più naturale e meno ripetitiva.

la Figura 3.5 mostra alcuni esempi di responses definite per il chatbot implementato.

```
responses:
  utter_default:
    - text: "❓ Sorry, I didn't understand that. Could you please rephrase your request?"
    - text: "⚠️ I'm not sure what you mean. Can you try saying it in a different way?"

  utter_greet:
    - text: "👋 Hello! I'm your book recommendation assistant. I can help you find books in two ways:
      \n• Search by title - Tell me the book title you're looking for
      \n• Discover with filters - I'll suggest books based on genre, author, rating, number of pages, and more!"
    - text: "👋 Welcome! I'm here to help you find the perfect book. You can:
      \n• Search for a specific title if you know what you're looking for
      \n• Use filters (genre, author, rating, number of pages) to discover new books"

  utter_goodbye:
    - text: "😊 Happy reading! Come back anytime you need a book recommendation.
      \nType \"new search\" to start a new conversation."
    - text: "😊 Goodbye! Enjoy your book and see you next time!
      \nType \"new search\" to find another book."
```

Figure 3.5: Esempi di responses definite nel file `domain.yml`

Tutte le responses definite sono organizzate in tre categorie:

- **Gestione della conversazione:** `utter_greet`, `utter_goodbye`, `utter_default`, `utter_stop`.
- **Gestione della ricerca:** `utter_ask_newsearch`, `utter_ask_more_info`, `utter_no_results`, `utter_ask_another`.
- **Feedback di ricerca:** `utter_ask_is_this_correct`, `utter_book_found_happy`, `utter_book_notfound`.

Ogni response prevede 1-2 varianti alternative per aggiungere varietà al dialogo. I messaggi più importanti, come `utter_greet` e `utter_ask_newsearch`, forniscono anche istruzioni chiare sulle modalità di utilizzo del bot, distinguendo tra ricerca per titolo specifico e ricerca tramite filtri.

3.5 Stories: percorsi conversazionali del Bot

Le **stories** rappresentano percorsi conversazionali di esempio che descrivono sequenze tipiche di interazioni tra utente e bot. Ogni story è composta da una serie di steps che alternano gli **intents dell'utente** (ciò che l'utente dice) e le **azioni del bot** (come il

bot risponde). Durante la fase di training, il modello di dialogue management apprende da queste stories per predire l'azione successiva più appropriata in base al contesto conversazionale. Le stories sono probabilistiche: il modello le utilizza per generalizzare su conversazioni simili mai viste prima, senza dover definire esplicitamente ogni possibile percorso. Le stories sono particolarmente utili per modellare flussi complessi e multi-turno, dove l'ordine delle azioni e il contesto sono determinanti. La Figura 3.6 mostra alcune stories significative implementate per questo chatbot, evidenziando i pattern conversazionali.

```

- story: search by author
  steps:
    - intent: find_book_by_author
      entities:
        - author
      - action: action_search_books

- story: search by title
  steps:
    - intent: find_book_by_title
      entities:
        - title
      - action: action_search_books

- story: user wants completely new search
  steps:
    - intent: new_search
    - action: action_reset_slots_with_message
    - action: utter_ask_newsearch

- story: user searches title and confirms it's correct
  steps:
    - intent: find_book_by_title
      entities:
        - title
    - action: action_search_books
    - slot_was_set:
      - found_specific_book: true
    - action: utter_ask_is_this_correct
    - intent: affirm
    - action: utter_book_found_happy
    - action: action_reset_slots

- story: user searches title but it's wrong
  steps:
    - intent: find_book_by_title
      entities:
        - title
    - action: action_search_books
    - slot_was_set:
      - found_specific_book: true
    - action: utter_ask_is_this_correct
    - intent: deny
    - action: utter_book_notfound
    - action: action_reset_slots
  
```

Figure 3.6: Esempi di stories definite nel file *stories.yml*

Nella parte sinistra dell'immagine sono rappresentate stories per la ricerca diretta di libri:

- **search by author:** l'utente esprime l'intent `find_book_by_author` specificando un autore tramite l'entity `author`, e il bot risponde eseguendo la custom action `action_search_books` che interroga il dataset e restituisce i risultati filtrati.
- **search by title:** pattern analogo per la ricerca per titolo, dove l'entity `title` viene estratta e utilizzata per filtrare i libri.
- **user wants completely new search:** questa story gestisce il caso in cui l'utente vuole azzerare i filtri precedenti. All'intent `new_search` segue l'azione `action_reset_slots_with_message` che resetta tutti gli slots e conferma l'operazione, seguita da `utter_ask_newsearch` che richiede come procedere con la nuova ricerca.

Nella parte destra sono mostrate stories più articolate che gestiscono scenari di feedback:

- **user searches title and confirms it's correct:** rappresenta un flusso positivo dove l'utente cerca un libro per titolo, il sistema lo trova tramite `action_search_books`, lo slot `found_specific_book` viene impostato a `true`, il bot chiede conferma con `utter_ask_is_this_correct`, l'utente risponde affermativamente e il bot conclude con `utter_book_found_happy` seguito dal reset degli slots.
- **user searches title but it's wrong:** gestisce invece il caso negativo, dove dopo la richiesta di conferma l'utente nega (`deny`) che il libro trovato sia corretto. Il bot risponde con `utter_book_notfound` per esprimere empatia e invita a riprovare, concludendo con il reset degli slots.

3.6 Rules: gestione di comportamenti fissi

Le **rules** sono regole deterministiche che definiscono comportamenti obbligatori e invarianti del bot in risposta a specifici intents o condizioni. A differenza delle stories, che sono probabilistiche e permettono al modello di generalizzare, le rules garantiscono che determinate azioni vengano sempre eseguite in situazioni ben definite, indipendentemente dal contesto conversazionale precedente.

La Figura 3.7 mostra le rules implementate per questo chatbot.

```
rules:
- rule: Say goodbye anytime the user says goodbye
  steps:
    - intent: goodbye
    - action: utter_goodbye

- rule: Handle stop intent
  steps:
    - intent: stop
    - action: utter_stop

- rule: Reset slots on new search
  steps:
    - intent: new_search
    - action: action_reset_slots_with_message
    - action: utter_ask_newsearch

- rule: User selects a book from the list
  steps:
    - intent: select_book
    - action: action_show_selected_book
  wait_for_user_input: false

- rule: Ask user to rephrase when not understood
  steps:
    - intent: nlu_fallback
    - action: action_default_fallback
```

Figure 3.7: Rules definite nel file `rules.yml`

Le rules gestiscono comportamenti che devono essere sempre deterministici:

- **Say goodbye anytime the user says goodbye:** quando l'utente esprime l'intent `goodbye`, il bot deve sempre rispondere con `utter_goodbye`, indipendentemente dalla fase della conversazione.
- **Handle stop intent:** l'intent `stop` rappresenta una richiesta esplicita di interrompere l'operazione corrente. La rule assicura che venga sempre eseguita la response `utter_stop` per confermare l'interruzione.
- **Reset slots on new search:** questa rule è fondamentale per gestire il comando "new search". Ogni volta che l'utente esprime l'intent `new_search`, il sistema deve obbligatoriamente azzerare tutti gli slots attraverso `action_reset_slots_with_message` e successivamente chiedere come procedere con `utter_ask_newsearch`.
- **User selects a book from the list:** quando viene presentata una lista di libri e l'utente seleziona un elemento specifico tramite l'intent `select_book`, la rule attiva `action_show_selected_book` che mostra i dettagli del libro scelto.
- **Ask user to rephrase when not understood:** gestisce il fallback quando il modello NLU non riesce a classificare con sufficiente confidenza l'intent dell'utente (`nlu_fallback`). In questo caso viene sempre eseguita `action_default_fallback`, che risponde con un messaggio di incomprensione e invita l'utente a riformulare la richiesta.

È importante notare che alcune rules, come quella per l'intent `goodbye` e `new_search`, sono presenti anche come stories. In questi casi, la rule rinforza il comportamento deterministico garantendo che l'azione venga sempre eseguita, mentre le stories forniscono esempi di contesto per il training del modello. Rasa gestisce correttamente questa sovrapposizione, dando priorità alle rules quando applicabili.

3.7 Actions e reset degli slots

Le **actions** rappresentano la logica applicativa del chatbot, ovvero il codice Python che viene eseguito quando il bot deve compiere operazioni complesse che non possono essere gestite con semplici responses statiche. Le actions permettono di interrogare database, manipolare dati, eseguire calcoli e generare messaggi dinamici personalizzati in base al contesto conversazionale. Nel caso di questo chatbot, le actions gestiscono tre aspetti fondamentali: la **ricerca e il filtraggio dei libri** nel dataset, la **visualizzazione dei dettagli dei libri**, e il **reset dello stato conversazionale**. Tutte le actions sono implementate nel file `actions.py` e vengono eseguite da un action server separato che comunica con il core di Rasa tramite API REST.

La Figura 3.8 mostra l'elenco delle actions definite per il bot sviluppato.

```
actions:
- action_search_books
- action_reset_slots
- action_reset_slots_with_message
- action_show_selected_book
- action_default_fallback
```

Figure 3.8: *Lista delle actions definite*

Le actions implementate sono descritte di seguito, con particolare attenzione alla loro logica implementativa.

action_search_books

Questa è l'action centrale del chatbot, responsabile di filtrare il dataset dei libri in base ai criteri specificati dall'utente. L'implementazione segue questi step:

1. **Estrazione degli slots:** recupera tutti i criteri di ricerca memorizzati (**genre**, **author**, **title**, **num_pages**, **min_pages**, **max_pages**, **min_rating**, **characters**).
2. **Filtraggio progressivo:** parte dal dataframe completo e applica sequenzialmente ogni filtro attivo utilizzando *pandas*. Ad esempio, per il genere utilizza *str.contains()* con case-insensitive matching, mentre per le pagine applica operatori di confronto numerico.
3. **Gestione caso speciale titolo:** quando viene specificato un titolo, l'action cerca di restituire un singolo libro specifico (il più popolare per numero di recensioni) invece di una lista, impostando lo slot **found_specific_book** a **true**.
4. **Ordinamento dei risultati:** i libri trovati vengono ordinati per numero di recensioni e rating, privilegiando i titoli più popolari e meglio valutati.
5. **Formattazione dell'output:** viene distinto nel caso di una lista di 3-5 libri o un singolo libro nel caso di output finale.
6. **Memorizzazione risultati:** salva la lista dei libri suggeriti nello slot **suggested_books** per permettere successivamente di recuperare i dettagli quando l'utente clicca un bottone.
7. **Gestione dell'assenza di risultati:** se nessun libro corrisponde ai filtri, genera un messaggio user-friendly che descrive in linguaggio naturale i criteri applicati.

action_reset_slots

Action silenziosa che azzerata tutti gli slots senza inviare messaggi all'utente. Viene utilizzata al termine di un flusso conversazionale completato con successo (ad esempio dopo che l'utente ha confermato di aver trovato il libro cercato).

action_reset_slots_with_message

Versione esplicita dell'azione di reset, che oltre ad azzerare gli slots invia un messaggio di conferma all'utente ("Starting a fresh search! What are you looking for?"). Viene invocata quando l'utente richiede esplicitamente di iniziare una nuova ricerca tramite l'intent `new_search`. La distinzione tra reset silenzioso ed esplicito migliora l'esperienza utente, fornendo feedback solo quando necessario.

action_show_selected_book

Gestisce la selezione di un libro specifico dalla lista di risultati precedentemente mostrata. L'implementazione segue questi passi:

1. **Estrazione dell'indice:** recupera lo slot `book_index` oppure, se non presente, lo estrae dal testo del messaggio utente tramite espressione regolare.
2. **Validazione e recupero dati:** verifica che l'indice sia valido e accede allo slot `suggested_books` che contiene la lista serializzata dei risultati precedenti.
3. **Visualizzazione completa:** mostra tutti i dettagli del libro selezionato (descrizione, anno, generi completi) e imposta `found_specific_book` a `true`.

Questa action permette una navigazione fluida dai risultati generali ai dettagli specifici senza dover rieseguire la query sul dataset.

action_default_fallback

Gestisce i casi in cui il modello NLU non riesce a classificare con sufficiente confidenza l'intent dell'utente. Invia uno dei messaggi definiti in `utter_default` (selezionato casualmente tra le varianti disponibili) e restituisce l'evento `UserUtteranceReverted()`, che rimuove l'ultimo messaggio utente dalla storia conversazionale. Questo evita che un messaggio incompreso contamini il tracker e influenzi negativamente le predizioni successive.

4 Setup Telegram e avvio del Bot

Il chatbot è stato integrato con **Telegram**, una delle piattaforme di messaggistica istantanea più diffuse, che offre un'API completa e ben documentata. Telegram fornisce diversi vantaggi per lo sviluppo e il deployment di chatbot: supporto nativo per bottoni interattivi, invio di media e documenti, gestione di comandi personalizzati, e soprattutto un processo di configurazione semplificato attraverso **BotFather**, il bot ufficiale di Telegram dedicato alla creazione e gestione di altri bot. Inoltre, Rasa offre un connettore nativo per Telegram, rendendo l'integrazione diretta e senza necessità di sviluppare codice aggiuntivo per la comunicazione con le API della piattaforma.

Questo capitolo descrive il processo di configurazione completo: dalla creazione del bot con BotFather all'ottenimento delle credenziali, dalla configurazione del connettore Telegram in Rasa alle procedure di avvio dei servizi necessari.

4.1 Installazione dell'ambiente di sviluppo

Prima di procedere con la configurazione del Bot su Telegram, è necessario preparare l'ambiente di sviluppo locale installando Python, Rasa e le dipendenze necessarie. Si consiglia di utilizzare **Miniconda**, una versione minimale del gestore di pacchetti Conda, che permette di creare ambienti virtuali isolati per gestire le dipendenze del progetto senza interferire con altre installazioni Python presenti nel sistema.

- **Installazione di Miniconda:** Scaricare l'installer da <https://www.anaconda.com/docs/getting-started/miniconda/main> ed eseguirlo.
- **Aggiungere Miniconda al PATH di sistema:** su Windows, selezionare "Add Miniconda to my PATH" durante l'installazione, oppure usare sempre Miniconda Prompt.
- **Creazione ambiente Rasa:** creare un ambiente con Python 3.8 attraverso il comando:

```
conda create -n rasa python=3.8
conda activate rasa
```

- **Installazione delle dipendenze:** eseguire il comando:

```
pip uninstall-y psychopg2-binary
pip install psychopg2-binary==2.9.9
pip install rasa
```

- **Installazione delle dipendenze aggiuntive:** per poter lavorare con il dataset nelle actions, è necessario installare "pandas" con il comando `pip install pandas`.

4.2 Creazione del Bot con BotFather

Il primo passo per rendere operativo il chatbot su Telegram consiste nella registrazione del Bot sulla piattaforma e nell'ottenimento delle **credenziali di autenticazione** necessarie per la comunicazione con le API di Telegram. Questo processo viene gestito attraverso BotFather, il bot ufficiale di Telegram dedicato alla creazione e configurazione di nuovi Bot.

Il processo di creazione è semplice, basta avviare una conversazione cercando @BotFather su Telegram e seguire i seguenti passaggi:

1. Inviare il comando `/newbot`.
2. Specificare un username univoco che termini con "bot" (es. "Biblio2026_bot").

Al termine della procedura, BotFather conferma la creazione del bot e fornisce un **token di autenticazione (HTTP API token)**, una stringa alfanumerica univoca che identifica il bot e permette l'accesso alle API di Telegram. Questo token deve essere mantenuto segreto e non condiviso, in quanto chiunque ne entri in possesso può controllare completamente il bot.

La Figura 4.1 mostra il flusso di creazione del Bot attraverso la conversazione con BotFather.

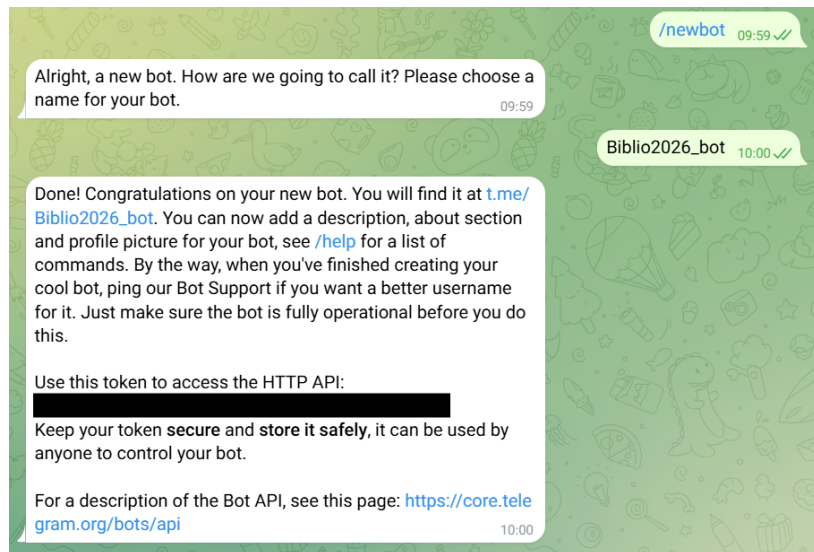


Figure 4.1: *Processo di creazione di "Biblio2026_bot" tramite BotFather*

4.3 Configurazione di ngrok per il webhook

Per permettere a Telegram di comunicare con il Bot Rasa in esecuzione su una macchina locale, è necessario esporre il server Rasa a Internet attraverso un URL pubblico raggiungibile. La soluzione consiste nell'utilizzo di **ngrok**, un servizio di tunneling che crea un canale sicuro tra Internet e il localhost, fornendo un URL pubblico temporaneo che reindirizza il traffico verso la macchina locale.

Telegram utilizza il meccanismo dei webhook per inviare i messaggi al bot: quando un utente scrive al bot, Telegram effettua una richiesta **HTTP POST** verso l'URL del webhook configurato, contenente i dati del messaggio. Rasa riceve questa richiesta e processa il messaggio, generando una risposta che viene inviata nuovamente a Telegram tramite API. Senza un URL pubblico accessibile, Telegram non può consegnare i messaggi al bot.

Il processo di configurazione di ngrok si articola nei seguenti passaggi:

1. **Download:** scaricare ngrok da <https://ngrok.com/> ed estrarre l'eseguibile.
2. **Avvio del tunneling:** digitare su terminale il comando `"ngrok http 5005"`. Ngrok genera un URL pubblico HTTPS visibile che andrà incollato nel file `credentials.yml`. Questa operazione va eseguita ogni volta che si vuole utilizzare il Bot su Telegram.

```
* One gateway for every AI model. Available in early access *now*: https://ngrok.com/i/ai
Session Status      online
Account             matteojesi21@gmail.com (Plan: Free)
Update              update available (version 3.35.0, Ctrl-U to update)
Version             3.24.0-msix
Region              Europe (eu)
Latency              445ms
Web Interface        http://127.0.0.1:4040
Forwarding           https://ares-keyed-allyson.ngrok-free.dev -> http://localhost:5005
Connections
  ttl   opn   rt1   rt5   p50   p90
    0    0    0.00  0.00  0.00  0.00
```

Figure 4.2: Connessione con ngrok con url evidenziato

Dunque, l'url pubblico da dover usare nel file `credentials.yml` è, in questo caso, quello evidenziato nella Figura 4.2.

4.4 Configurazione del file `credentials.yml`

Una volta ottenuti il token di autenticazione da BotFather e l'URL pubblico del web-hook tramite ngrok, è necessario configurare Rasa per abilitare la comunicazione con Telegram. Questa configurazione avviene attraverso il file `credentials.yml`, che contiene le credenziali e i parametri di connessione per tutti i canali di messaggistica supportati da Rasa. Questo file si trova nella directory principale del progetto Rasa e definisce, per ciascun canale attivo, le informazioni necessarie per stabilire la connessione. Nel caso di Telegram, è necessario specificare il token di autenticazione, lo username del bot e l'URL del webhook dove Rasa riceverà i messaggi. La configurazione del connettore Telegram richiede tre parametri fondamentali:

telegram:

```
access_token: "YOUR_TELEGRAM_TOKEN"
verify: "YOUR_BOT_USERNAME"
webhook_url: "YOUR_NGROK_URL/webhooks/telegram/webhook"
```

Di seguito vengono descritti in dettaglio i parametri da configurare e il loro significato:

- **access_token**: token di autenticazione fornito da BotFather. Stringa alfanumerica che inizia con l'ID numerico del bot seguito da caratteri casuali. Funge da chiave di accesso alle API Telegram e deve rimanere segreto. Reperibile con il processo illustrato in Figura 4.1.
- **verify**: username del Bot (es. Biblio2026_bot). Deve corrispondere esattamente al nome registrato su Telegram, senza il simbolo @. Utilizzato da Rasa per verificare l'autenticità delle richieste.
- **webhook_url**: URL pubblico completo dove Telegram invia i messaggi. Reperibile con il processo illustrato nella Sezione 4.3.

4.5 Avvio del sistema

Completata la configurazione di **BotFather**, **ngrok** e del file `credentials.yml`, è possibile avviare il sistema completo e rendere il bot operativo su Telegram. L'architettura di Rasa richiede l'esecuzione simultanea di tre processi distinti, ciascuno con una funzione specifica: il Rasa server per la gestione del dialogo, l'Action server per l'esecuzione delle custom actions, e il tunnel ngrok per esporre il server a Internet.

Prima di avviare i servizi, è necessario preparare l'ambiente aprendo tre terminali separati, di cui due necessariamente utilizzando Miniconda Prompt o Anaconda Prompt). In questi due terminali si deve attivare l'ambiente virtuale Rasa con il comando:

```
conda activate rasa
```

Successivamente si deve navigare nella directory del progetto contenente i file di configurazione Rasa.

Nel terminale rimanente, avviare ngrok per creare il tunnel verso la porta 5005, con il comando:

```
ngrok http 5005
```

Questo terminale deve rimanere aperto per tutta la durata dell'utilizzo del bot, poiché la chiusura interrompe il tunnel e rende il bot irraggiungibile da Telegram.

In uno degli altri due terminali con l'ambiente Rasa, digitare:

```
rasa run --enable-api --cors "*" --debug
```

Questo comando avvia il server Rasa e può impiegare qualche secondo.

Nel rimanente terminale, infine, avviare il server delle custom actions con:

```
rasa run actions
```

Con tutti e tre i servizi attivi, il Bot è completamente operativo e accessibile su Telegram. Per testarlo:

1. Aprire l'app Telegram (mobile o desktop).
2. Cercare il bot utilizzando l'username configurato (es. @Biblio2026_bot).
3. Avviare la conversazione.
4. Interagire con il Bot inviando messaggi.

5 Test del Bot

In questo capitolo vengono descritti i test eseguiti sul bot conversazionale, con l'obiettivo di verificare il corretto funzionamento delle principali modalità di ricerca e il comportamento del sistema durante l'interazione con l'utente.

5.1 Funzionalità verificate

Il bot consente all'utente di cercare un libro tramite uno o più dei seguenti criteri:

- Genere
- Titolo
- Autore
- Personaggi
- Numero di pagine
- Valutazione

Inoltre, quando l'utente fornisce un titolo completo (o sufficientemente preciso), il bot restituisce immediatamente il libro corrispondente se presente nel database.

5.2 Impostazione dei test

I test sono stati svolti simulando conversazioni complete (richiesta → risposta del bot → eventuale affinamento), verificando:

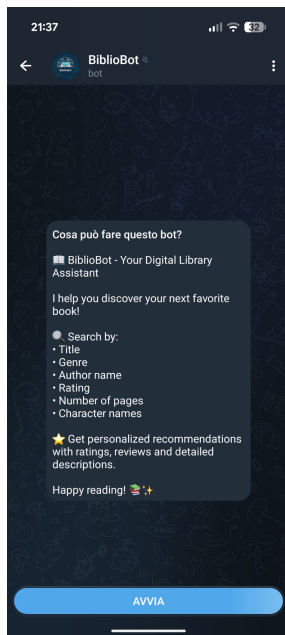
- corretto riconoscimento del criterio di ricerca espresso dall'utente;
- corretta interrogazione del database e coerenza dei risultati restituiti;
- chiarezza della risposta (informazioni essenziali e proposta di alternative quando necessario).

5.3 Casi di test eseguiti

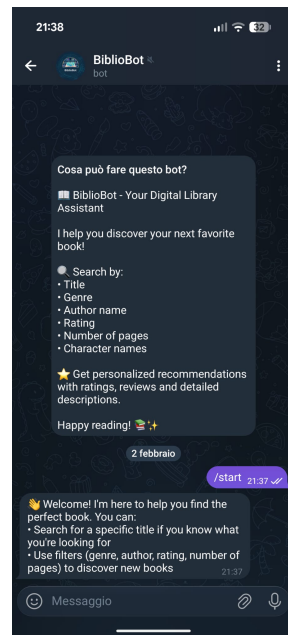
5.3.1 Avvio

All'apertura dell'applicazione, il bot visualizza un messaggio introduttivo (Fig. 5.1a) che riassume in modo sintetico le funzionalità disponibili e il tipo di supporto che può offrire all'utente durante la ricerca di un libro.

Selezionando il pulsante *Avvia*, la conversazione viene avviata: il bot si presenta e fornisce le prime indicazioni operative per guidare l'utente nella formulazione della richiesta (Fig. 5.1b).



(a) *Descrizione iniziale*



(b) *Avvio della conversazione*

5.3.2 Ricerca per titolo

Quando l'utente inserisce un titolo esatto, il bot verifica la presenza del libro nel database. In caso di riscontro positivo, viene mostrata una schermata dedicata (Fig. 5.2) contenente le principali informazioni di dettaglio, tra cui autore, numero di pagine, valutazione (rating), genere e una breve descrizione dell'opera.

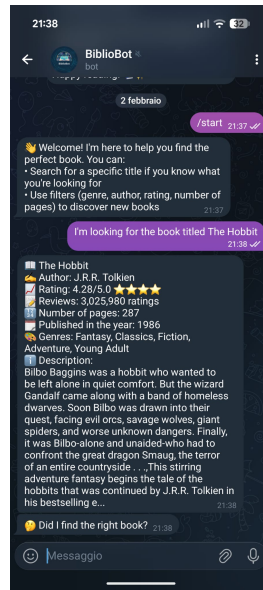
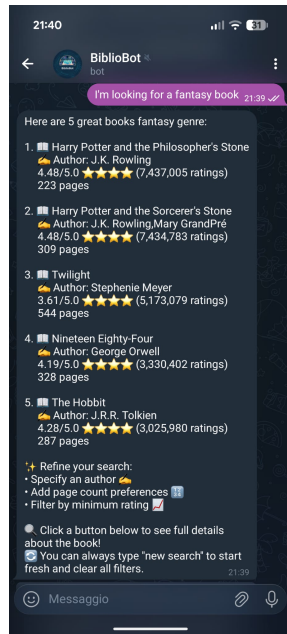


Figure 5.2: Risultato della ricerca per titolo

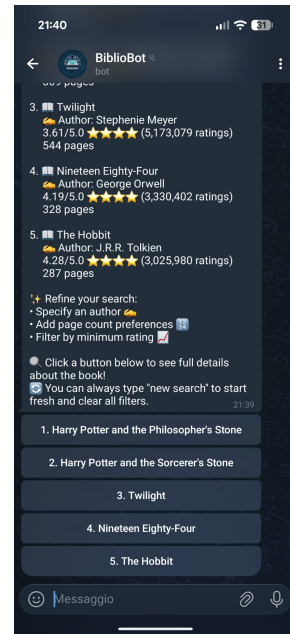
5.3.3 Ricerca per filtri

Oltre alla ricerca diretta per titolo, l'utente può individuare un libro combinando più filtri, in modo da restringere progressivamente l'insieme dei risultati. In particolare, il bot supporta i seguenti criteri: **genere**, **autore**, **numero di pagine** (minimo, massimo o intervallo) e **valutazione** (minima, massima o intervallo).

All'impostazione di ciascun filtro, il bot aggiorna la risposta mostrando i **cinque libri più pertinenti** rispetto ai vincoli selezionati, includendo per ogni risultato alcune informazioni essenziali (ad esempio titolo, autore e dettagli principali). Come mostrato in Fig. 5.3a e Fig. 5.3b, l'utente può iniziare applicando un filtro per genere e proseguire aggiungendo ulteriori vincoli.



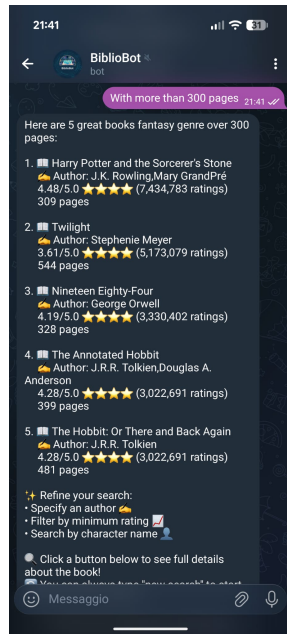
(a) Impostazione del filtro per genere



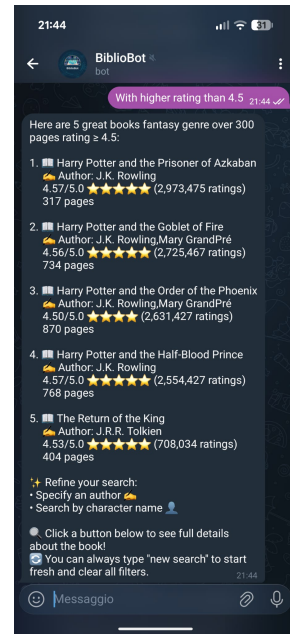
(b) Risultati della ricerca

Figure 5.3: Esempio di ricerca tramite filtro per genere

La stessa logica viene applicata anche agli altri filtri: il numero di pagine (Fig. 5.4a) e la valutazione (Fig. 5.4b) consentono di selezionare libri coerenti con preferenze quantitative, mentre la ricerca per autore (Fig. 5.5a) e per personaggi (Fig. 5.5b) permette di orientare la selezione sulla base di elementi narrativi o dell'autorialità.

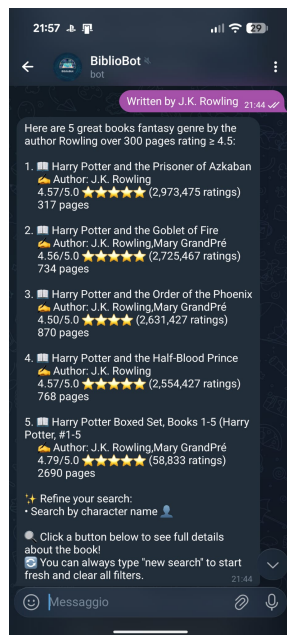


(a) Filtro per numero di pagine

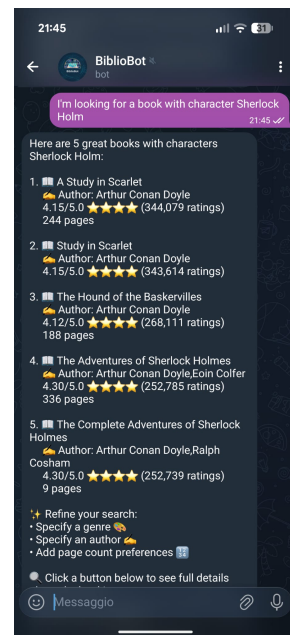


(b) Filtro per valutazione (rating)

Figure 5.4: Esempi di ricerca con filtri quantitativi



(a) Filtro per autore



(b) Filtro per personaggi

Figure 5.5: Esempi di ricerca con filtri testuali

Infine, una volta visualizzata la lista dei risultati, l'utente può selezionare uno dei libri proposti tramite i pulsanti associati ai risultati mostrati dal bot. In qualunque

momento è inoltre possibile avviare una **nuova ricerca** (Fig. 5.6), azzerando i filtri precedentemente impostati.

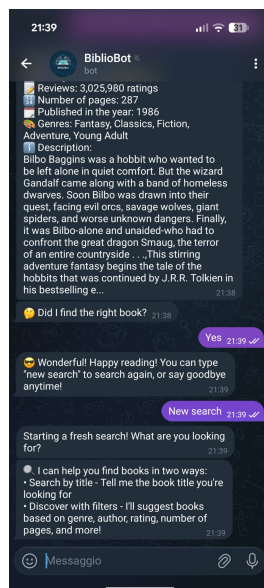


Figure 5.6: *Avvio di una nuova ricerca*

5.4 Conclusioni sui test

I test effettuati hanno permesso di verificare il corretto funzionamento delle principali funzionalità del bot e la coerenza del comportamento nei diversi scenari di ricerca. In particolare, è stato confermato che il sistema supporta in modo efficace sia la ricerca diretta per titolo, con restituzione immediata del libro quando presente nel database, sia la ricerca guidata tramite filtri (genere, autore, personaggi, numero di pagine e valutazione), anche in combinazione tra loro.

Durante le prove è stata inoltre valutata la capacità del bot di gestire richieste incomplete o ambigue, proponendo alternative e suggerimenti utili per proseguire la ricerca, e di reagire correttamente nei casi di assenza di risultati. Nel complesso, l'attività di testing ha evidenziato una buona stabilità del flusso conversazionale e un'esperienza d'uso coerente con l'obiettivo del progetto: supportare l'utente nell'individuazione rapida di libri di interesse, fornendo indicazioni chiare e risultati pertinenti.